

SaaN 2L-GRL: Two-Level Graph Representation Learning Empowered With Subgraph-as-a-Node

Jeong-Ha Park , Bo-Young Lim , Kisung Lee , and Hyuk-Yoon Kwon , *Member, IEEE*

Abstract—In this study, we propose a novel graph representation learning (GRL) model, called Two-Level GRL with Subgraph-as-a-Node (SaaN 2L-GRL in short), that partitions input graphs into smaller subgraphs for effective and scalable GRL in two levels: 1) local GRL and 2) global GRL. To realize the two-level GRL in an efficient manner, we propose an abstracted graph, called Subgraph-as-a-Node Graph (SaaN in short), to effectively maintain the high-level graph topology while significantly reducing the size of the graph. By applying the SaaN graph to both local and global GRL, SaaN 2L-GRL can effectively preserve the overall structure of the entire graph while precisely representing the nodes within each subgraph. Through time complexity analysis, we confirm that SaaN 2L-GRL significantly reduces the learning time of existing GRL models by using the SaaN graph for global GRL, instead of using the original graph, and processing local GRL on subgraphs in parallel. Our extensive experiments show that SaaN 2L-GRL outperforms existing GRL models in both accuracy and efficiency. In addition, we show the effectiveness of SaaN 2L-GRL using diverse kinds of graph partitioning methods, including five community detection algorithms and representative edge- and vertex-cut algorithms.

Index Terms—Graph partitioning, graph representation learning, learning efficiency, representation accuracy, subgraph-as-a-node, two-level architecture.

I. INTRODUCTION

A. Backgrounds

GRAPHS have been widely used to represent complex relationships between objects effectively [1]. A graph consists of nodes, the fundamental units, and edges that represent connections between two nodes [2]. Real-world applications using graphs encompass diverse domains, including social networks, road networks, autonomous driving systems, and web graphs, where graph structures can represent relationships (i.e., edges) between different objects (i.e., nodes). Many organizations, such

as Airbnb, NBC News, and NASA, use graph databases to manage their businesses [3]. In the healthcare industry, AstraZeneca analyzes graph data to find patients with similar travel patterns and health conditions [4]. The U.S. Army also uses a graph database to manage and track equipment maintenance across thousands of vehicles, parts, and locations worldwide [5].

Graph representation learning (GRL) has become a useful solution for graph-based mining and machine learning, effectively addressing various tasks like node classification, link prediction, and node clustering [6], [7]. GRL converts graph data into low-dimensional vector spaces to process and analyze complex graph structures. Given real-world graphs' scale and inherent complexity, it is challenging to maintain both relationships between nodes (i.e., local features) and their topological structures (i.e., global features) during the learning process.

In this paper, we focus on the deep learning-based GRL approach, which has shown promising results across various graph analysis tasks. The deep learning-based GRL models can be categorized into 1) random walk-based and 2) graph neural network-based models [8]. Random walk-based models [9], [10], [11], such as DeepWalk [10] and Node2Vec [11], make node sequences by randomly exploring nodes within the graph and applying probability models. However, as random walks only capture the local patterns of nodes, they cannot effectively represent the global structure of the entire graph [12]. Moreover, they cannot consider additional information, such as node attributes or edge weights. As a result, graph neural networks have emerged, with the capacity to capture both global structures and additional features [13], [14], [15]. Although they improve the representation accuracy of GRL, they require significant computation costs due to the inherent complexity of neural network-based models.

Efficient processing is another essential requirement for large-scale GRL. Parallel processing can speed up computation by dividing the entire graph into subgraphs and concurrently processing them. Therefore, we employ graph partitioning to address scalability issues inherent in large-scale graphs. In this study, we categorize graph partitioning algorithms into deterministic and non-deterministic algorithms based on whether they generate the subgraphs with a predetermined number of partitions. For example, conventional graph partitioning algorithms, including edge-cut and vertex-cut algorithms, are classified as *deterministic partitioning*, where the number of partitions is predetermined. In contrast, community detection algorithms are classified as *non-deterministic partitioning* since they find the optimal number of partitions during execution.

Manuscript received 4 January 2024; revised 18 May 2024; accepted 22 June 2024. Date of publication 11 July 2024; date of current version 13 November 2024. This work was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) under Grant 2022R1F1A1067008 and in part by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education under Grant 2019R1A6A1A03032119. Recommended for acceptance by H. Hu. (Corresponding author: Hyuk-Yoon Kwon.)

Jeong-Ha Park, Bo-Young Lim, and Hyuk-Yoon Kwon are with the Graduate School of Data Science, Seoul National University of Science and Technology, Seoul 01811, South Korea (e-mail: pjh4709@seoultech.ac.kr; esob618@seoultech.ac.kr; hyukyeon.kwon@seoultech.ac.kr).

Kisung Lee is with the Division of Computer Science and Engineering, Louisiana State University, Baton Rouge, LA 70803 USA (e-mail: klee76@lsu.edu).

Digital Object Identifier 10.1109/TKDE.2024.3421933

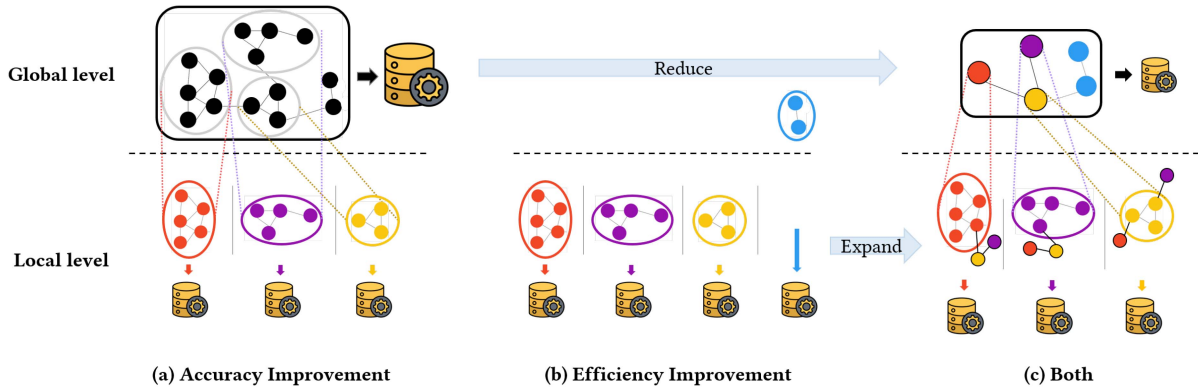


Fig. 1. The basic concept of Two-Level GRL with Subgraph-as-a-Node (SaaN 2L-GRL).

B. Main Idea

In this paper, we introduce a novel graph representation learning (GRL) model, *Two-Level GRL with Subgraph-as-a-Node (SaaN 2L-GRL)*, designed to enhance both the accuracy of representation and the efficiency of learning. This model strategically partitions the entire graph into subgraphs to facilitate 1) local GRL within each subgraph for capturing intricate intra-subgraph relationships and 2) global GRL across subgraphs to understand topological structures between them. This structure allows for parallel processing of subgraphs, significantly improving learning efficiency.

Fig. 1 shows the architecture of Two-Level GRL and the concept of SaaN 2L-GRL. Fig. 1(a) illustrates Accurate 2L-GRL, a primitive method aiming to maximize accuracy by focusing on major subgraphs for local GRL and the original graph for global GRL. However, this method considerably increases learning time due to the simultaneous execution of local and global GRL. Conversely, Fig. 1(b) shows another primitive method, Efficient 2L-GRL, which prioritizes efficiency by processing all subgraphs in local GRL regardless of their size and eliminating global GRL. While efficient, this method tends to overlook the overall structure, resulting in decreased GRL accuracy.

Fig. 1(c) illustrates the main idea of our proposed model, SaaN 2L-GRL, which combines the accuracy of Accurate 2L-GRL with the efficiency of Efficient 2L-GRL. To effectively capture the overall graph patterns without using the original graph, we introduce the *Subgraph-as-a-Node Graph (SaaN)*, where each subgraph is represented as a representative node. This allows us to effectively represent relationships between subgraphs and nodes, significantly reducing computational overhead. The SaaN graph is applied at both global and local levels. Globally, it replaces the original graph while preserving the overall structure of the graph. Locally, it maintains inter-node relationships within and across subgraphs. As the SaaN graph is much smaller than the original, it incurs minimal overhead.

C. Highlights

We summarize four salient points of our proposed model:

- 1) Unlike previous studies that focused solely on improving either accuracy or efficiency, our model enhances both by

using a SaaN graph to abstract the entire graph into a more condensed unit. This structure allows for precise capturing of intricate relationships within subgraphs and preserves topological structures between them while also increasing efficiency through parallel processing.

- 2) Our model can be seamlessly integrated with any existing GRL model as the base model, requiring no specific modifications or dependencies. Previous studies [16], [17], [18], [19] have utilized community information to enhance representation accuracy, but these were limited to specific types of GRL models, namely random walk-based or graph neural network-based models. In contrast, our model is versatile, proving effective for both random walk-based and GNN-based models, as demonstrated by our experimental results.
- 3) We address the limitations of current deep learning-based GRL models by substituting the entire graph with a SaaN graph to capture global structures better and segmenting large-scale graphs into manageable sub-problems for concurrent processing, thus reducing time complexity.
- 4) The effectiveness of our model does not depend on the type of graph partitioning algorithm used. It adapts well to both deterministic and non-deterministic methods, utilizing several community detection algorithms [20], [21], [22], [23], [24] and deterministic methods such as Metis [25] and Libra [26].

To validate the effectiveness of the proposed model, we conduct both time complexity and empirical analysis.

- 1) *Time complexity analysis:* We confirm that SaaN 2L-GRL improves the time complexity from $O(|E|)$ to $O(|E'|)$ by leveraging a SaaN graph instead of the original graph, where E and E' are the edges for the original and SaaN graph, respectively. This is a remarkable improvement because $|E'|$ is only 10.33 ~ 18.29% of $|E|$ in our evaluated datasets.
- 2) *Empirical analysis:* Through extensive experiments using several real-world datasets, we show the superiority of our proposed model compared with a total of eight existing GRL models in terms of accuracy and efficiency. SaaN 2L-GRL significantly improves the AUC value of the existing GRL models and reduces their learning time in both

link prediction and node classification tasks. Furthermore, our model demonstrates its extended effectiveness on various graph partitioning algorithms, including Metis and Libra.

The rest of this paper is organized as follows. In Section II, we describe the related work. In Section III, we formalize our problem. In Section IV, we explain our motivation. In Section V, we describe the proposed model. In Section VI, we show the experimental results. Finally, we conclude the paper and discuss future work in Section VII.

II. RELATED WORK

A. Graph Representation Learning

The GRL models can be categorized as follows according to the embedding output [27]: 1) node embedding, 2) edge embedding, 3) subgraph embedding, and 4) whole graph embedding. Each category is based on the unit used for embedding and is preferred for specific downstream tasks. For example, node embedding is advantageous for node-centric graph analysis tasks, such as node classification and clustering. As another example, previous studies [28], [29] produced graph embeddings to calculate the similarity between a given pair of graphs in the graph matching task. Although the appropriate embedding type depends on the downstream tasks, node embedding is considered the most fundamental and essential because other embedding types can be built on top of it. Therefore, in this study, we focus on node embedding and demonstrate its applications to both node- and edge-based analysis.

To generate node embeddings, several techniques have been developed: 1) matrix factorization, 2) deep learning, 3) edge reconstruction, and 4) generative models [27]. In this paper, we focus on node embedding using deep learning-based models, which can be categorized into two types: 1) random walk-based and 2) graph neural network-based models [8]. Random walk-based models [30] generate node sequences starting from random initial nodes and learn these sequences for each node. DeepWalk [10], the first random walk-based model, applied the skip-gram of Word2Vec to the node sequences based on random walks [9]. Node2Vec [11] modified DeepWalk to use a more flexible sampling strategy. While these random walk-based methods take advantage of utilizing the local connectivity structures, which are relatively simple and easy to implement, they have limitations in capturing the global structures of the graph.

Graph neural network (GNN) models have been used to apply the existing neural networks to graph data such as CNN, attention network, and reinforcement learning. Kipf et al. [13] proposed graph convolutional networks (GCNs), which consider both graph structures and node labels based on layer-wise propagation. Since then, GCNs-based variants have emerged: GraphSAGE [14], which samples neighboring nodes to reduce the learning overhead of GCNs, and GAT [15], which applies a self-attention mechanism to enhance the interpretability and performance of the model. Furthermore, GNN-based methods have come in many forms, such as recurrent neural networks [31],

[32], [33], autoencoders [34], [35], and generative models [36], [37].

Recently, there has been an increasing focus on self-supervised learning for GNN, aiming to learn graph data representations independent of labeled data. This approach learns from the graph data itself through various ways such as predicting the presence or absence of edges between nodes [38], [39], reconstructing the graph from randomly masked nodes or edges [40], [41], or contrastive methods [42], [43], [44], [45], [46]. Zhu et al. [43] proposed a graph contrastive representation learning method with adaptive augmentation, where new augmentation schemes based on node centrality are designed to highlight important connections and corrupt unimportant node features. Wang et al. [45] proposed a novel supervised contrastive graph learning method, ClusterSCL, which retains the structures and attribute properties of a graph in the form of nodes' cluster distributions.

It is worth noting that several existing studies have introduced a concept similar to our SaaN, but with different purposes. Sun et al. [47] proposed SUGAR, an end-to-end graph classification method based on subgraph embedding. It sampled multiple subgraphs using BFS traversal based on degree and reconstructed a reduced graph, transforming some subgraphs into supernodes while preserving the structural information of the subgraphs. Kim et al. [48] proposed Subgraph-To-Node translation to focus on efficient subgraph-level tasks using pre-defined subgraphs (e.g., sets of proteins in a protein interaction network). Ying et al. [49] introduced a hierarchical graph representation learning method with a differentiable graph pooling module, which learns cluster assignments for nodes at each layer and forms coarsened inputs for the next layer. However, these studies targeted small-sized graphs, with an average of 68 nodes, such as molecules, whereas we deal with node embedding on much larger graphs, such as social networks and citation networks. Furthermore, they focused on subgraph-level or graph-level tasks, not applicable to node-level tasks, which are our main target problems.

A few studies have explored parallel processing in GRL. He et al. [50] extended MILE [51], a multi-level embedding framework that coarsens a large network using structural similarity by implementing parallel processing and distributed training. Yang et al. [52] developed a distributed graph embedding system, EDGES, which uses GPU clusters to train the model on large graphs in parallel. Bohm et al. [53] proposed Multi-coreGEMPE, an efficient algorithm that uses parallel processing on multiple CPU cores to construct the node embeddings by compressing the adjacency matrix. While these methods significantly reduced the learning time for graph embedding, they did not focus on representation accuracy. In contrast, we aim to improve both representation accuracy and learning efficiency.

We note that several research attempts have been made to improve both the efficiency and accuracy of node embeddings using matrix factorization, such as NetSMF [54] and ProNE [55]. However, to the best of our knowledge, no existing deep learning-based methods have focused on improving both the accuracy and efficiency of GRL.

B. Graph Partitioning

Graph partitioning divides an original graph into smaller subgraphs and allows for parallel processing on subgraphs, which can significantly reduce the computational time complexity compared to processing on the entire graph. There are two mainstream graph partitioning algorithms: 1) edge-cut [25], [56], [57], [58], which cuts edges, evenly partitioning vertices, and 2) vertex-cut [26], [59], which cuts vertices, evenly partitioning edges while replicating cut vertices. Metis [25] is the representative edge-cut algorithm, which decomposes the graph into subgraphs with evenly distributed vertices while minimizing discarded edges. The recursive partitioning process in a multilevel way is quite effective, even if it may lead to memory and computational load issues. Libra is a state-of-the-art algorithm for vertex-cut [26], which assigns edges to partition with the lowest workload, considering the vertices associated with the edges. It employs a greedy heuristics-based approach to minimize the replication factor. In this study, we refer to this kind of partitioning method as *deterministic partitioning*. Because it requires the number of partitions as a hyperparameter before execution, it conducts partitioning based on this deterministic value.

With the emergence of more advanced graph partitioning methods, Kong et al. [60] proposed a clustering-based streaming graph partitioning architecture designed for handling large-scale web graphs with parallelism. Zhou et al. [61] proposed an efficient and adaptive graph partitioning method for geo-distributed data centers. Yin et al. [62] proposed a new subgraph-based representation learning on large-scale graphs by extracting subgraphs through random walks and processing them in parallel. All these methods aimed to reduce communication costs and achieve load balancing among processing units through uniformed graph partitioning, resulting in significantly reduced processing time.

C. Community Detection

Community detection primarily aims to identify communities of densely interconnected nodes within a graph [63]. Community detection can be regarded as a specific case within edge-cut graph partitioning, as it involves splitting the graph by cutting edges while avoiding node replication. Community information is essential for understanding the global structures of a graph, and thus, numerous community detection algorithms have been developed. Raghavan et al. [20] proposed a label propagation-based algorithm, Blondel et al. [21] proposed a multilevel modularity optimization algorithm, Newman et al. [22] developed an eigenvector matrix-based algorithm, and Rosvall et al. [24] proposed an infomap algorithm minimizing the expected length of random walker trajectories. In this paper, we call partitioning by community detection *non-deterministic partitioning* because it finds the optimal number of partitions during execution.

Community detection has been applied to various problems, such as recommendation, information diffusion, and event prediction [64], [65], [66]. Park et al. [67] used community detection to identify relevant groups associated with cyberattack events and made predictions of cyberattacks from the most relevant

community. In addition, community detection has received considerable attention from other applications, such as user behavior analysis [68] and heterogeneous question answering [69].

Several research efforts have attempted to enhance node embedding by incorporating community information [16], [17], [18], [19]. Cavallari et al. [16] proposed a framework for joint node- and community-level embedding with the help of community detection. Liu et al. [19] proposed a node embedding method, AnECI, that preserves community information using a graph convolutional encoder with a new modularity function. Tu et al. [12] proposed community-enhanced NRL (CNRL) that uses community information to embed local node characteristics and global community patterns jointly. While these methods share a common feature with ours in the case where community detection is used as our partitioning method, our model has three key advantages over previous methods. First, we utilize community information not only to enhance the quality of node embeddings but also to physically partition the graph based on this information, enabling efficient learning. Second, our method is compatible with any existing GRL model as the base model. CNRL [12] can also adopt the base model, but it only applies to random walk-based models, whereas our model applies to both random walk-based and GNN-based models. Finally, our method can apply the aforementioned methods as the base GRL model to improve its performance further. We report our experimental results using CNRL as our base model in Section VI.

In our previous work [70], we only focused on community detection as the graph partitioning unit. In this study, we extensively extend the previous approach with the following four points. 1) We accommodate both deterministic, including conventional edge-cut and vertex-cut algorithms, and non-deterministic graph partitioning algorithms, including community detection. 2) Through motivating experiments, we validate our idea to simultaneously enhance accuracy and efficiency through the SaaS graph. 3) We enrich our experiments by covering representative graph partitioning and various community detection algorithms. We also demonstrate the scalability and parallelism of our method by scaling out the dataset size and CPU cores. 4) We conduct a theoretical analysis of the time complexity involved in both local and global GRLs of our model.

D. Summary

To clearly show our purpose and strengths compared to existing relevant approaches, we summarize their distinct characteristics in Table I. Table I compares our method with hierarchical GRL [49] and community-based GRL [12], [19]. We select the key comparison characteristics as follows: 1) downstream tasks, 2) partitioning objectives, 3) partitioning methods, 4) GRL requirement on the full graph, and 5) expected outcomes. They commonly deal with graph partitioning but with different objectives. Compared to hierarchical GRL, our method aims to target node classification and link prediction based on node-level embedding and partition the graph, which is compatible with both community detection and graph partitioning methods, achieving both accuracy and efficiency improvement. Compared

TABLE I
COMPARISON OUR METHOD WITH HIERARCHICAL GRL [49] AND
COMMUNITY-BASED GRL [12], [19]

	Hierarchical GRL [49]	Community-based GRL [12, 19]	Ours
Downstream tasks	Graph classification	Node classification or Link Prediction	
Partitioning objectives	Integrating nodes to subgraphs by clustering	Guiding the node embedding by the underlying community structure	Finding an effective subgraph that abstracts the entire graph into a more condensed unit.
Partitioning methods	Clustering	Community detection	Community detection or Graph partitioning method
GRL requirement on the full graph		Yes	No
Expected outcomes		Improvement of accuracy	Improvement of accuracy and efficiency

to community-based GRL, we can utilize arbitrary graph partitioning methods and do not require GRL on the full graph, achieving efficiency and further optimizing its accuracy.

III. PROBLEM DEFINITION

In this section, we define the problem for graph representation learning (GRL) dealt with in this study. We are given an input graph $G = (V, E)$, where V is a set of nodes, and E is a set of edges, with $(v_i, v_j) \in E$ indicating an edge between $v_i (\in V)$ and $v_j (\in V)$. We cover both cases with and without node attributes, addressing both random walk-based and GNN-based models. Specifically, G has an attribute matrix A with the shape of $(|V| * L)$, where $|V|$ and L denote the numbers of nodes and node attributes, respectively. Edge weights are not considered in our study. GRL transforms each node in G as a low dimensional vector denoted as $\mathbf{v}_i \in \mathbb{R}^d$, in which d represents the dimension of a representation space $\mathbb{R}$, by learning the features in G . GRL learns a mapping function $f(v_i) = \mathbf{v}_i$, which converts each node v_i into d -dimensional vector $\mathbf{v}_i$. As output, we obtain an embedding matrix with the shape of $(|V| * d)$ consisting of the results of learning each $\mathbf{v}_i$ in V .

We partition the entire graph G into k subgraphs $S = \{S_1, \dots, S_k\}$ according to partitioning strategy. Here, we note that depending on the partitioning method, overlapping nodes among the subgraphs may exist. Specifically, some nodes are overlapped across subgraphs for the vertex-cut partitioning method, while the edge-cut, including community detection, divides all nodes disjointly. In addition, there could be numerous small subgraphs, each containing only a few nodes, which are ineffective for GRL due to insufficient structural information. Therefore, we differentiate subgraphs that meet a specified size criterion and those that do not. More precisely, based on a predetermined threshold for the subgraph size, ρ , we define a set of major subgraphs $S_{major} = \{S_1, \dots, S_m\}$, where each subgraph S_i in S_{major} satisfies $|S_i| \geq \rho$, and a set of minor subgraphs $S_{minor} = \{S_{m+1}, \dots, S_k\}$ that are subgraphs not belonging to S_{major} , in which $|S_i|$ indicates the number of nodes in subgraph S_i .

IV. MOTIVATION

In this section, we describe the motivation of our proposed model by demonstrating the feasibility and benefits of our SaaN 2L-GRL. Most real-world graphs, such as social networks,

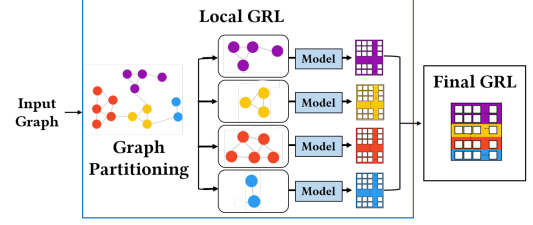


Fig. 2. Efficient 2L-GRL.

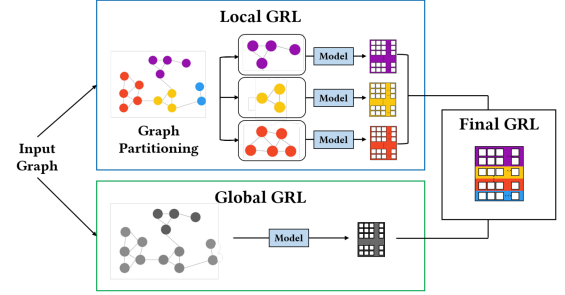


Fig. 3. Accurate 2L-GRL.

follow a power-law distribution, consisting of a few major subgraphs and many minor subgraphs. Because non-deterministic partitioning, such as community detection, finds the optimal subgraphs considering the closeness between nodes, it can deal with power-law distribution relatively well. Therefore, to cover such challenging graph structures, we first devise a method targeting community detection in this section and extend it to encompass deterministic partitioning.

As a first intuitive approach, we develop *Efficient 2L-GRL* that runs GRL on each subgraph (i.e., local GRL) independently, as illustrated in Fig. 2. This method is an implementation of the idea of Fig. 1(b) mentioned in Section I. We obtain the final GRL results by merging all the results of the local GRL. One main advantage of Efficient 2L-GRL is improved learning efficiency because we can parallelize local GRL on subgraphs using multiple cores/machines. However, Efficient 2L-GRL can have sub-optimal learning accuracy because it is hard to learn meaningful graph representations on tiny subgraphs and ignores inter-subgraph relationships.

To address the limitation, we present another approach, called *Accurate 2L-GRL*, that handles major and minor subgraphs differently, as illustrated in Fig. 3. This method corresponds to the idea of Fig. 1(a) mentioned in Section I. Accurate 2L-GRL runs local GRL on each major subgraph and global GRL on the original graph. To obtain the final GRL results, Accurate 2L-GRL integrates local GRL results for major subgraphs' nodes and global GRL results for minor subgraphs' nodes. Accurate 2L-GRL aims to improve learning accuracy by utilizing subgraph information in two levels: intra-subgraph relationships through local GRL and inter-subgraph relationships through global GRL. However, regarding learning efficiency, Accurate 2L-GRL has significant overhead because it requires global GRL on the original graph.

TABLE II
MOTIVATING EXPERIMENTAL RESULTS

Metric	Accuracy			Learning Time (sec.)		
	Node2vec	GraphSAGE	CNRL	Node2vec	GraphSAGE	CNRL
Baseline	0.8454	0.8478	0.8033	5844	6282	6034
Accurate 2L-GRL	0.9267	0.9497	0.8891	7074	8023	7449
Efficient 2L-GRL	0.8221	0.8321	0.7824	3476	2666	3996

To validate the feasibility of utilizing subgraph information for GRL, we run the two approaches (i.e., Accurate 2L-GRL and Efficient 2L-GRL) using three baselines (i.e., Node2vec, GraphSAGE, and CNRL) for link prediction. Table II summarizes the results on the Foursquare graph dataset (further described in Section VI). The results demonstrate reduced learning time of Accurate 2L-GRL (but at the cost of slightly reduced accuracy) and improved learning accuracy of Efficient 2L-GRL (but at the cost of higher learning time). Motivated by these results, in this paper, we propose SaaN 2L-GRL that can take advantage of the two quite different approaches.

V. PROPOSED MODEL

A. The Concept

In this study, we propose a new GRL model, called Two-Level GRL with *Subgraph-as-a-Node* (simply, *SaaN 2L-GRL*). To maximize both accuracy and efficiency, SaaN 2L-GRL aims to maintain the advantages of parallel processing in Efficient 2L-GRL while preserving the accuracy of Accurate 2L-GRL. To achieve this, we introduce a *Subgraph-as-a-Node Graph* (simply, *SaaN*) (See Definition V.1) that abstracts each subgraph as a representative node. The SaaN graph effectively captures the relationships between subgraphs and other (minor) subgraphs. Importantly, by employing SaaN graphs at both the local and global levels, we can effectively represent the relationships between subgraphs and nodes while significantly reducing the learning overhead compared to the original graph. We emphasize that our SaaN covers any kind of partitioning methods, both deterministic and non-deterministic algorithms. We actually deal with five diverse community detection algorithms, Metis for a conventional graph edge-cut algorithm, and Libra for a conventional graph vertex-cut algorithm in the experiments.

Definition V.1 (Subgraph-as-a-Node Graph (SaaN)): The SaaN graph replaces each major subgraph S_m of the original graph G with super-node s_m . The k th node attribute of s_m in the SaaN graph is aggregated using a function $\text{CONDENSE}(A_{m,k})$, where $A_{m,k}$ is a set of k th node attribute values for all nodes in S_m . $\text{CONDENSE}(A_{m,k})$ returns a single numerical value using one of Cases V.1 ~ V.3, defined below, according to the type of node attributes. An edge between two super-nodes s_p and s_q is created if there is at least one edge between nodes in S_p and S_q . Similarly, an edge between a super-node s_t and a node n_s not in S_{minor} is created if there is at least one edge between nodes in S_t and n_s .

Case V.1 (Numerical attributes): A numerical attribute is defined by numerical values (e.g., word embedding vector, latitude/longitude, and age). For a given numerical attribute $A_{m,k}$,

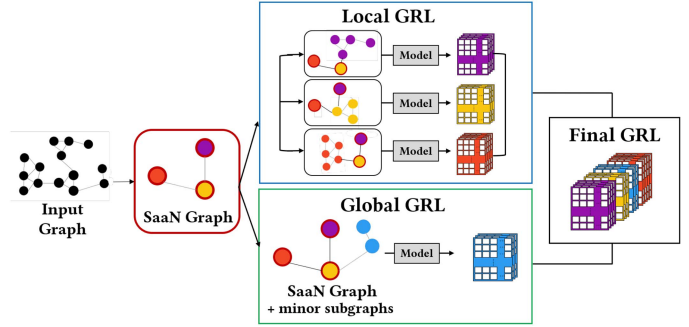


Fig. 4. SaaN 2L-GRL.

$\text{CONDENSE}(A_{m,k})$ is defined as the mean of the values in $A_{m,k}$.

Case V.2 (Categorical attributes): A categorical attribute is defined by categorical values (e.g., country code and gender). For a given categorical attribute $A_{m,k}$, $\text{CONDENSE}(A_{m,k})$ is defined as the mode value of the values in $A_{m,k}$.

Case V.3 (Network attributes): A network attribute is defined by the graph topology (e.g., node degree and node centrality). Network attributes are prioritized over numerical and categorical attributes. For a given network attribute $A_{m,k}$, $\text{CONDENSE}(A_{m,k})$ is defined as a summated value of the values in $A_{m,k}$. $\square$

Fig. 4 shows the overall structure of SaaN 2L-GRL. The core idea is to construct the SaaN graph from the input graph and utilize it at both local and global levels. In the example in Fig. 4, the original graph consists of three major subgraphs and blue nodes belonging to minor subgraphs. The SaaN graph effectively maintains the connection between subgraphs without managing the node-level relationship in the original graph. The SaaN graph is used to preserve global structural information for global GRL. In local GRL, super-nodes in the SaaN graph are added to each subgraph, allowing each subgraph to be learned while preserving connections with relevant subgraphs. Within our proposed framework, arbitrary base GRL models, such as Node2Vec and GraphSAGE, can be used for executing the global and local GRLs, labeled as “Model” in the figure. Specifically, for global GRL, the model is applied to a SaaN Graph connected with minor subgraphs. For local GRL, the model is applied for each local subgraph connected with the SaaN graph. This base model functions as a modular component that converts the given input graph into node embeddings independent of the specific base model. Finally, we combine the results from both levels to generate a final embedding for the input graph. In this way, we leverage the subgraph from both semantic and physical perspectives through the SaaN graph at each level, improving representation accuracy and learning efficiency at the same time.

B. Global GRL

In global GRL, we aim to capture the global structure of the entire graph. To achieve this, we preserve global relationships at the subgraph level using the SaaN graph, instead of the original graph. Only with the tiny size of the SaaN graph, we

can effectively retain the global structure of the original graph. Furthermore, we effectively capture the relationship between the super-nodes in the SaaN graph and nodes in minor subgraphs, which cannot be effectively learned in local GRL due to its limited size. Consequently, global GRL with the SaaN graph can improve the representation accuracy while significantly reducing learning overhead. Notably, although vertex-cut algorithms generate the duplication for the cut vertices, this duplication does not affect global GRL because each subgraph is abstracted as a super-node. We outline the procedure for running the global GRL, as indicated in (1), and any GRL model can be utilized for $GRL()$.

$$GlobalGRL(G) = GRL(SaaN(G) \cup \sum_{i=m+1}^k S_i) \quad (1)$$

C. Local GRL

In local GRL, we aim to capture the elaborate differences in the relationships between nodes in a subgraph. To prevent loss of connections with other subgraphs during this process, we enhance local GRL with the SaaN graph by connecting the super-nodes in the SaaN graph to the nodes in each subgraph. Here, we exclude the super-node, s_i , corresponding to the subgraph S_i . This results in a minimal overhead for the SaaN graph while further improving the representation accuracy. We formulate the process of constructing local GRL as shown in (2), and any GRL model can be employed for $GRL()$.

$$LocalGRL(G) = \sum_{i=1}^m GRL(SaaN(G) \setminus s_i \cup S_i) \quad (2)$$

It is worthwhile to mention that, for the vertex-cut partitioning algorithms, a node may overlap across multiple subgraphs. In such instances, we independently obtain each local GRL as normal and subsequently obtain the overlapped node's representation through post-processing methods for local GRLs. Specifically, the node representation vector for overlapped nodes is determined with the following methods: 1) a singular value of the randomly selected node (Rand), 2) a singular value of the node in a subgraph with the highest degree (HighDeg), and 3) the average of the learned representation vectors for overlapped nodes (Avg). We measure the performance of SaaN 2L-GRL in the experiments by using these three methods.

D. Algorithm

Algorithm 1 shows the processing steps of SaaN 2L-GRL. Initially, we partition the entire graph G is partitioned into subgraphs $S = \{S_1, \dots, S_k\}$ (line 1). This step is commonly processed for any base model $GRL()$. Then, it constructs G_{SaaN} by converting each major subgraph in G into a single super-node (line 2). Here, we define the value for each k th node attribute $A_{m,k}$ of the super-node s_m , i.e., $s_{m,k}$, according to cases V.1, V.2, and V.3 in Definition V.1 (lines 3–14). In global GRL, we learn the node embedding on G_{SaaN} , including nodes in minor subgraphs. The result vectors for nodes belonging to the minor subgraph are stored in $\mathbf{v}_{V_{minor}}$ (lines 15–16). Then,

Algorithm 1: SaaN 2L-GRL.

Input: Graph G , partitioning strategy P , base model $GRL()$

/* 1. SaaN Graph Construction */

- 1: According to P , partition G to $S = \{S_1, \dots, S_k\}$
- 2: Generate G_{SaaN} by converting subgraph S_m in S_{major} into a single super-node s_m
- 3: **for** $S_m \in S_{major}$ **do** */* Condensing node attributes */*
- 4: **for** $A_{m,k} \in S_m$ **do**
- 5: **switch** $TYPE(A_{m,k})$ **do**
- 6: NETWORK ATTRIBUTE
- 7: $s_{m,k} = SUM(A_{m,k})$; **break**
- 8: NUMERICAL ATTRIBUTE
- 9: $s_{m,k} = MEAN(A_{m,k})$; **break**
- 10: CATEGORICAL ATTRIBUTE
- 11: $s_{m,k} = MODE(A_{m,k})$; **break**
- 12: **end switch**
- 13: **end for**
- 14: **end for**

/* 2. Global GRL with SaaN */

- 15: $V_{minor} = \{v_i, \forall v_i \in S_{minor}\}$
- 16: $\mathbf{v}_{V_{minor}} = GRL(G_{SaaN} \cup V_{minor})$

/* 3. Local GRL with SaaN */

- 17: **for** $S_m \in S_{major}$ **do** */* Parallel processing */*
- 18: $V_{S_m} = \{v_j, \forall v_j \in S_m\}$
- 19: $\mathbf{v}_{V_{S_m}} = GRL((G_{SaaN} \setminus s_m) \cup V_{S_m})$
- 20: **end for**
- 21: **for** $v_l \in$ overlapped nodes **do**
- 22: $\mathbf{v}_l = AGGREGATE(\mathbf{v}_l, \forall \mathbf{v}_l \in S)$
- 23: **end for**

/* 4. Final GRL */

- 24: Concatenate $\mathbf{v}_{V_{minor}}$ and $\sum_{i=1}^m \mathbf{v}_{V_{S_m}}$ to $GRL(G)$

Output: An embedding matrix with shape $(|V| * d)$

in local GRL, each major subgraph is augmented with the SaaN graph. We then perform the local GRL for major subgraphs in parallel. The result vectors for nodes belonging to the major subgraph are stored in $\mathbf{v}_{V_{S_m}}$ (lines 17–20). In the case of the vertex-cut algorithms, to deal with overlapped nodes, we aggregate the representation vectors of an overlapped node from the subgraphs where the node exists (lines 21–23) according to one of the three methods mentioned in Section V-C. Finally, we integrate the results of local and global GRL to obtain the final embedding (line 24).

Fig. 5 illustrates the process for generating the final graph representation using SaaN 2L-GRL. Consider an example where the input graph $G = \{n_1, n_2, \dots, n_{100}\}$ contains three major subgraphs: $S_1 = \{n_1, \dots, n_{30}\}$, $S_2 = \{n_{31}, \dots, n_{60}\}$, and $S_3 = \{n_{61}, \dots, n_{90}\}$, along with nodes $\{n_{91}, \dots, n_{100}\}$ in minor subgraphs. The SaaN graph G_{SaaN} consists of three super-nodes s_1, s_2 , and s_3 , each corresponding to the major subgraphs S_1, S_2 , and S_3 , respectively.

In the global GRL phase, we train G_{SaaN} to acquire a node embedding vector with a shape of $(13 * d)$. Subsequently, we extract the vector values specifically related to nodes

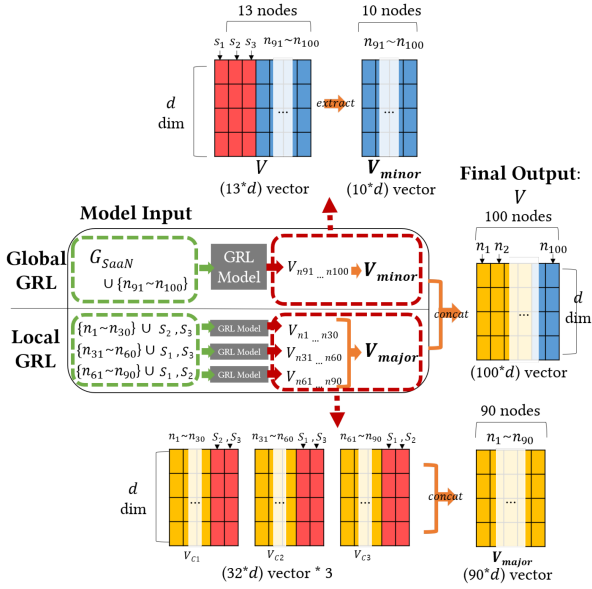


Fig. 5. Graph representation using SaaN 2L-GRL.

TABLE III
TIME COMPLEXITY COMPARISON FOR EACH METHOD BY LEVEL

Methods	Local GRL	Global GRL
Accurate 2L-GRL	$O(\frac{ E }{k})$	$O(E)$
Efficient 2L-GRL	$O(\frac{ E }{k})$	-
SaaN 2L-GRL	$O(\frac{ E + E' }{k})$	$O(E')$

$\{n_{91}, \dots, n_{100}\}$ to obtain the final embedding. In the local GRL phase, we learn major subgraphs in parallel. We augment the super-nodes in the SaaN graph to the nodes in each subgraph. That is, for a subgraph S_1 , we consider not only the nodes $\{n_1, \dots, n_{30}\}$ in S_1 but also super-nodes s_2 and s_3 to consider the relationship from nodes in S_1 . We learn the nodes and super-nodes and obtain a node embedding vector of shape $(32 * d)$. We then extract only the vector values corresponding to nodes $\{n_1, \dots, n_{30}\}$ in S_1 to obtain the final embedding.

E. Time Complexity Analysis

Given a graph $G = (V, E)$ with the number of nodes $|V|$ and the number of edges $|E|$, we assume that the numbers of subgraphs and computing nodes are equal to k . We analyze the time complexity for our final proposed model, SaaN 2L-GRL, by two levels: 1) local GRL and 2) global GRL. We omit the time complexity of graph partitioning because it is highly dependent on the algorithm used. Furthermore, the partitioning time is generally negligible compared to the GRL time, which is demonstrated in the experimental results.

To provide a baseline for comparison, we also consider two primitive models, Accurate 2L-GRL and Efficient 2L-GRL, which represent the upper and lower bound of the methods for SaaN 2L-GRL, respectively. Table III shows the analyzed results.

TABLE IV
REAL NETWORK DATASET STATISTICS

Dataset	# Nodes	# Edges	# Labels	# Attributes
Foursquare	114,324	607,333	N/A	5
Flickr	7,575	239,738	9	12,047
PubMed	19,717	44,338	3	500
CiteSeer	3,312	4,732	6	3,703
WikiCS	11,701	216,123	10	300
Coauthor-CS	18,333	163,788	15	6,805
Youtube _{small}	10,000	38,445	N/A	1
Youtube _{mid}	100,000	225,875	N/A	1
Youtube _{large}	1,000,000	2,328,165	N/A	1

Theorem 1: The time complexity for local GRL in Accurate 2L-GRL and Efficient 2L-GRL is $O(\frac{|E|}{k})$; that in SaaN 2L-GRL is $O(\frac{|E|+|E'|}{k})$.

Proof: The time complexity for local GRL in Accurate 2L-GRL, Efficient 2L-GRL, and SaaN 2L-GRL is determined by the number of edges in the largest subgraph they deal with because local GRLs are performed in parallel. Assuming that the edges are evenly distributed over subgraphs, the time complexity for local GRL in both Accurate 2L-GRL and Efficient 2L-GRL becomes $O(\frac{|E|}{k})$ because each subgraph has $\frac{|E|}{k}$ edges on average. In worst-case scenarios, such as with power-law graphs, the time complexity for local GRL is close to $O(|E|)$, meaning that it is determined between $O(\frac{|E|}{k})$ and $O(|E|)$ in practice. The time complexity for SaaN 2L-GRL becomes $O(\frac{|E|+|E'|}{k})$ because major subgraphs are augmented with the SaaN graph, which has the number of edges $|E'|$. ■

According to Theorem 1, we can observe that SaaN 2L-GRL has a higher overhead compared to Efficient 2L-GRL. However, because $|E'|$ in the SaaN graph is much smaller than $|E|$ in the original graph, the additional overhead is negligible. In the case of vertex-cut partitioning algorithms, additional time overhead in local GRL may occur due to duplicated nodes. However, this does not significantly affect the overall time complexity because the time complexity of GRL depends on $|E|$. In general, as evident from the experimental datasets in Table IV, the number of edges is notably larger than the number of nodes.

Theorem 2: The time complexity for global GRL in Accurate 2L-GRL is $O(|E|)$; that in SaaN 2L-GRL is $O(|E'|)$.

Proof: The time complexity for global GRL in Accurate 2L-GRL and SaaN 2L-GRL is determined by the number of edges in the graphs they deal with. Because Accurate 2L-GRL deals with the original graph for global GRL, its time complexity is $O(|E|)$. In contrast, SaaN 2L-GRL deals with a SaaN graph that has the number of edges $|E'|$, so its time complexity is $O(|E'|)$.

According to Theorem 2, in the case of Accurate 2L-GRL, we note that the time for global GRL occupies a significant portion of the total processes because its time complexity depends on the number of edges. While Efficient 2L-GRL does not require any cost for global GRL, SaaN 2L-GRL significantly reduces the overhead of Accurate 2L-GRL by dealing with a SaaN graph instead of the original graph. In summary, while Efficient 2L-GRL is the fastest, SaaN 2L-GRL becomes significantly faster than Accurate 2L-GRL because $|E'|$ is significantly smaller than $|E|$.

VI. PERFORMANCE EVALUATION

A. Experimental Settings

1) *Datasets*: To evaluate our proposed model, we utilize seven real-world datasets of different scales and characteristics, comprising three social networks (Foursquare, Flickr, and YouTube), two citation networks (PubMed and CiteSeer), one web knowledge graph (WikiCS), and one co-authorship graph (Coauthor-CS), as summarized in Table IV. They are homogeneous, undirected, and unweighted graphs with node attributes. For scalability experiments, we use three different scales of the YouTube dataset. We use graphs without node labels (marked with “N/A”) only for the link prediction task.

- *Foursquare*¹: This dataset provides a location-based social network service and allows users to “check-in” to a specific location or to communicate with other users. Node attributes in the dataset include the degree, country code, and category for the most visited venues by the user.
- *Flickr* [71]: This dataset is a photo-sharing website in Yahoo that provides photo storage and sharing services as well as online community services. Node attributes in the dataset include tag descriptions for each picture where the tag is shared by users.
- *PubMed* [72]: This dataset contains scientific publications from the PubMed database about diabetes, which are classified into one of three classes. Each publication in the dataset is described by a TF/IDF weighted word vector from a dictionary, which consists of 500 unique words.
- *Citeseer* [73]: This dataset contains scientific publications classified into one of six classes. Each publication in the dataset is described by a 0/1-valued word vector indicating the absence/presence of the corresponding word from a dictionary, which consists of 3,703 unique words.
- *WikiCS* [74]: This Wikipedia-based dataset is constructed from Wikipedia categories, specifically ten classes corresponding to branches of computer science. Node features are derived from the text of the corresponding articles and calculated as the average of pre-trained word embeddings in 300 dimensions.
- *Coauthor-CS* [75]: This dataset is a co-authorship graph based on the Microsoft Academic Graph from the KDD Cup 2016 challenge. Here, nodes represent authors connected by edges if they have a co-authored paper. Node features represent paper keywords for each author’s papers, and the node class is labeled as each author’s most active fields of study.
- *YouTube* [76]: This dataset consists of friendship groups and includes over 1.1 million users and 4.9 million links. We generate three datasets of different scales from the entire original data, $Youtube_{small}$, $Youtube_{mid}$, and $Youtube_{large}$. For each dataset, we randomly select n nodes from the top $5 \times n$ nodes with the highest degrees to generate graphs with sufficient relational information.

We conduct a scalability test using the YouTube datasets to evaluate the performance as the dataset size increases. For the

remaining datasets, we aim to cover all the base models for each downstream task by assigning them exclusively to two downstream tasks: link prediction and node classification. Here, we take into consideration certain constraints. For example, Foursquare cannot be applied for the node classification because node labels are not provided, and running base models such as AnECI and GCA leads to out-of-memory errors in some experimental settings on large datasets. Therefore, we perform experiments on Foursquare, CiteSeer, and WikiCS for link prediction and Flickr, PubMed, CiteSeer and Coauthor-CS for node classification, distributing eight base models across these datasets. We fix GraphSAGE in node classification while varying datasets to observe the variations resulting from using different datasets with the same base model. We also fix GAT while varying graph partitioning methods on two different datasets.

2) *Base Models*: We adopt a total of eight GRL models, including recently proposed models. Four widely used GRL models, Node2Vec [11], GCN [13], GAT [15], and GraphSAGE [14], were used. Two models that utilize community information for graph embedding, CNRL [12] and AnECI [19], were used to show their effectiveness as the base models for ours. Lastly, we used two self-supervised GRL models, GCA [43] and ClusterSCL [45], representing an emerging field in GRL.

3) *Experimental Environments*: We implement all codes using Python (version 3.6.13) and develop the parallel execution of our methods using the Python multiprocessing library. We implement several community detection algorithms and the SaaN graph generation using the iGraph Python library (version 0.9.9). We use the Word2Vec model provided by the Gensim (version 4.1.2) library for training Node2Vec, and the GraphSAGE, GCN, and GAT models provided by the StellarGraph library (version 1.2.1) implemented on TensorFlow Keras (version 2.6.2). We reproduce CNRL², AnECI³, GCA⁴ and ClusterSCL⁵ based on the codes provided by their authors. Our source code is publicly available at <https://github.com/parkjungha/Two-level-GRL>.

We use a machine equipped with an Intel Xeon Silver 4210R 20-core CPU @ 2.40 GHz and 32 GB RAM running on Ubuntu 18.04.6 LTS for all experiments. We use an NVIDIA GeForce RTX 3080 GPU for neural network-based model training.

4) *Evaluation Metrics*: In the experiments, we evaluate the performance of our model and base models in terms of accuracy and efficiency. To evaluate the accuracy of the representations, we utilize the AUC score [77] for link prediction and accuracy for node classification. We use the elapsed time for model training to assess the efficiency of learning. To consider the variations of each experiment, we execute each experiment three times and present its average and standard deviation.

5) *Parameter Settings*: Table V shows the hyperparameter values used in the experiments. We set the default number of CPU cores for parallel processing to 20.

To verify that our model is not dependent on a particular graph partitioning algorithm, we employ both partitioning

²http://nlp.csai.tsinghua.edu.cn/%7Etcc/datasets/simplified_CNRL.zip

³<https://github.com/Gmrylxb/AnECI>

⁴<https://github.com/CRIPAC-DIG/GCA>

⁵<https://github.com/wyl7/ClusterSCL>

¹<https://sites.google.com/site/yangdingqi/home/foursquare-dataset>

TABLE V
EXPERIMENTAL PARAMETERS

Parameters	Values		
GRL Models	Common	Embedding dimension (d)	256
		Size threshold (ρ)	100
		Number of cores (k)	20
	Random walk based methods	p	1.0
		q	1.0
		walk length	80
		window size	10
	GNN-based methods	Batch size	128
		Epochs	50
		Drop out rate	0.3
		Optimizer	Adam
		Learning rate	0.001

strategies: 1) non-deterministic partitioning and 2) deterministic partitioning. The non-deterministic partitioning methods involve five community detection algorithms: 1) label propagation (LP) [20], 2) multilevel modularity (MM) [21], 3) eigenvector-based (EG) [22], 4) random walk-based (RW) [23], and 5) infomap algorithms (IM) [24]. The deterministic partitioning methods involve representative partitioning algorithms for edge-cut and vertex-cut, namely, Metis [25] and Libra [26], respectively. We choose the label propagation algorithm as the default method for non-deterministic partitioning.

B. Performance on Link Prediction

Link prediction aims to predict the presence or absence of an edge between two nodes. With the node embeddings learned through GRL, we build a logistic regression model for random walk-based models or attach a fully connected layer to the last layer of GNN-based GRL models to predict the probability of edge existence for a given node pair (i.e., adopting diverse prediction models). For the learning phase, we create a training set by randomly eliminating 10% of the edges from the graph. Subsequently, the GRL models are trained using the remaining graph. The test set comprises the 10% removed edges as positive samples, along with an equal number of unconnected node pairs serving as negative samples.

Table VI summarizes link prediction results for eight base GRL models on the Foursquare, CiteSeer, and WikiCS datasets. The results demonstrate that our model not only can be equipped with diverse GRL models but also improves the representation accuracy while significantly reducing the learning time for each evaluated base GRL model. Specifically, our model enhances accuracy by a range of 1.35% to 12.47% and reduces learning time by 31.79% to 48.1% across various base models and datasets. Notably, the accuracy improvement achieved by our model with AnECI is comparatively modest compared to other base models. We assert that this is due to the fact that AnECI's embeddings already effectively retain community information acquired through GRL. Nevertheless, we note that, while our model shows a modest improvement in accuracy, it significantly reduces the learning time by 38.2%.

Fig. 6 shows the learning curves, in terms of the AUC value and loss, of our model and compares them with those of the base model. Due to space constraints, we only display the result of GraphSAGE on the Foursquare dataset. The results demonstrate

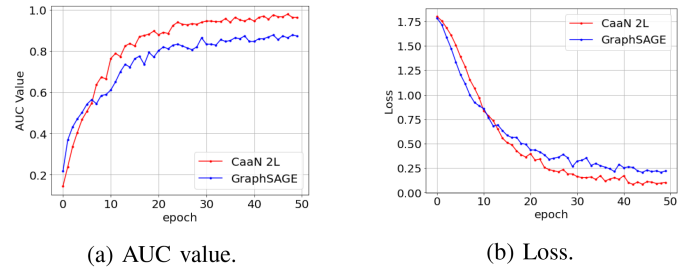


Fig. 6. Learning curve for link prediction.

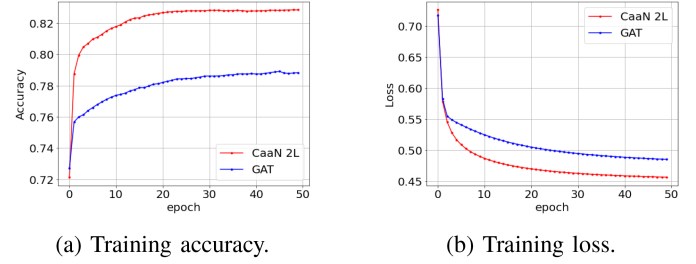


Fig. 7. Learning curve for node classification.

that our model converges to the higher AUC value (and smaller loss) at a faster pace than the base model.

C. Performance on Node Classification

Node classification aims to assign each node to one or multiple classes. The acquired node embeddings are fed into a logistic regression model for random walk-based models or a fully connected layer for GNN-based models for multi-label classification. During the learning process, 70% of nodes are randomly chosen as the training set, while the remaining nodes constitute the testing set. Table VII summarizes node classification results for eight base GRL models on the Flickr, PubMed, CiteSeer, and Coauthor-CS datasets. Similar to the results of link prediction, they consistently demonstrate that our model improves accuracy and significantly reduces learning time compared to base models.

Fig. 7 shows the learning curves of our model and compares them with those of the base model. Due to space constraints, we only display the result of GAT on the PubMed dataset. Similar to the results in Fig. 6, they demonstrate that our model converges to a higher accuracy (and smaller loss) at a faster pace than the base model.

Fig. 8 illustrates the performance across various datasets when using GraphSAGE as the base model. Consistent with previous results, our model demonstrates an accuracy enhancement ranging from 7.89% to 10.42%, along with a reduction in learning time by 25.71% to 38.35% when compared to the original GraphSAGE model.

D. Ablation Studies

To assess the impact of the SaaN graph at each level within our model, we perform ablation studies by eliminating SaaN

TABLE VI
PERFORMANCE COMPARISON ON LINK PREDICTION ACROSS BASE MODELS AND DATASETS

Dataset	Base model	Baseline		SaaN 2L		Improvements (%)	
		Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
Foursquare	Node2Vec	0.8491 $\pm$ 0.007	5844 $\pm$ 61	0.9092 $\pm$ 0.002	3750 $\pm$ 65	+5.83	-35.83
	GraphSAGE	0.8478 $\pm$ 0.002	6270 $\pm$ 70	0.9535 $\pm$ 0.004	3254 $\pm$ 83	+12.47	-48.10
	GCN	0.8353 $\pm$ 0.2	7041 $\pm$ 65	0.8922 $\pm$ 0.2	4803 $\pm$ 73	+6.81	-31.79
	CNRL	0.8033 $\pm$ 0.003	6034 $\pm$ 60	0.8440 $\pm$ 0.002	4028 $\pm$ 65	+5.07	-33.24
CiteSeer	GAT	0.9080 $\pm$ 0.002	667 $\pm$ 35	0.9545 $\pm$ 0.002	350 $\pm$ 41	+5.12	-47.53
	GCA	0.9250 $\pm$ 0.001	508 $\pm$ 35	0.9622 $\pm$ 0.001	329 $\pm$ 19	+4.02	-35.43
	AnECI	0.9345 $\pm$ 0.001	555 $\pm$ 30	0.9471 $\pm$ 0.001	343 $\pm$ 28	+1.35	-38.20
	ClusterSCL	0.8604 $\pm$ 0.0146	388 $\pm$ 33	0.9061 $\pm$ 0.0106	242 $\pm$ 13	+5.31	-37.55
WikiCS	GraphSAGE	0.8443 $\pm$ 0.0025	2285 $\pm$ 64	0.8668 $\pm$ 0.0053	1241 $\pm$ 58	+2.66	-45.68
	GCN	0.8018 $\pm$ 0.0083	2470 $\pm$ 50	0.8423 $\pm$ 0.0030	1650 $\pm$ 20	+5.05	-33.19

TABLE VII
PERFORMANCE COMPARISON ON NODE CLASSIFICATION ACROSS BASE MODELS AND DATASETS

Dataset	Base model	Baseline		SaaN 2L		Improvements (%)	
		Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
Flickr	Node2Vec	0.5645 $\pm$ 0.005	2990 $\pm$ 68	0.6329 $\pm$ 0.006	1773 $\pm$ 48	+12.12	-40.70
	GraphSAGE	0.6412 $\pm$ 0.002	3027 $\pm$ 64	0.7088 $\pm$ 0.002	1866 $\pm$ 59	+10.54	-38.35
	GCN	0.6125 $\pm$ 0.002	3569 $\pm$ 62	0.7201 $\pm$ 0.001	2042 $\pm$ 44	+17.56	-42.79
	AnECI	0.6318 $\pm$ 0.004	2986 $\pm$ 59	0.6892 $\pm$ 0.004	1763 $\pm$ 69	+9.09	-40.95
PubMed	GAT	0.7828 $\pm$ 0.004	904.11 $\pm$ 84	0.8194 $\pm$ 0.005	672.13 $\pm$ 88	+4.68	-25.66
	GCA	0.7957 $\pm$ 0.002	747.12 $\pm$ 40	0.8310 $\pm$ 0.003	485.22 $\pm$ 29	+4.44	-35.05
	AnECI	0.8010 $\pm$ 0.002	773 $\pm$ 61	0.8524 $\pm$ 0.003	502 $\pm$ 73	+6.42	-35.05
	ClusterSCL	0.8732 $\pm$ 0.0024	621 $\pm$ 3	0.8874 $\pm$ 0.0026	345 $\pm$ 1	+1.62	-44.44
CiteSeer	CNRL	0.6465 $\pm$ 0.003	730.27 $\pm$ 25	0.6892 $\pm$ 0.001	555.46 $\pm$ 18	+6.60	-23.94
	AnECI	0.7169 $\pm$ 0.002	404.15 $\pm$ 21	0.7358 $\pm$ 0.003	277.06 $\pm$ 40	+2.64	-31.45
Coauthor-CS	GraphSAGE	0.9241 $\pm$ 0.001	433 $\pm$ 91	0.9654 $\pm$ 0.0018	313 $\pm$ 45	+4.47	-27.71
	GCN	0.7005 $\pm$ 0.001	349 $\pm$ 6	0.8836 $\pm$ 0.0064	285 $\pm$ 3	+26.13	-18.34

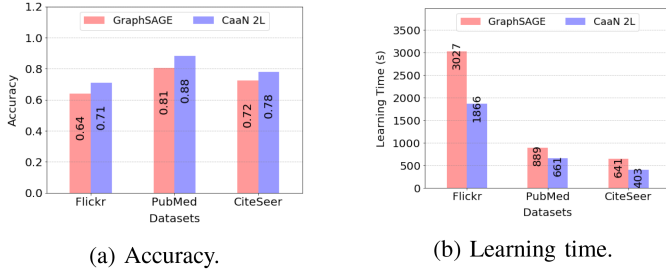


Fig. 8. Performance comparison for node classification on diverse datasets.

nodes separately at each level. When removing SaaN in local GRL, we refer to conducting local GRL on each subgraph solely using its original nodes, excluding super-nodes. When removing SaaN in global GRL, we indicate running global GRL on the original input graph. Additionally, we compare these results with the primitive methods (i.e., Accurate 2L-GRL and Efficient 2L-GRL).

Table VIII summarizes the accuracy and learning time results of our ablation studies. It demonstrates that our final model, as well as models without SaaN in each level, exhibit significantly improved accuracy compared to the baseline model. The results reveal the effectiveness of using SaaN for improving GRL accuracy, even with one-level adoption. The results also show that our final model has better accuracy than the models with one-level adoption. In particular, our final model is comparable to or even better than Accurate 2L-GRL in terms of accuracy, which maximizes the impact of the subgraph information without losing node-level information. In terms of learning time, our final model requires more computational costs than Efficient

TABLE VIII
ABLATION PERFORMANCE ANALYSIS

Task	Link prediction				Node classification			
	Foursquare		CiteSeer		PubMed		Flickr	
Base Model	GraphSAGE		Node2vec		GAT		GCN	
Metric	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
Baseline	0.8478 $\pm$ 0.002	6270 $\pm$ 70	0.9109 $\pm$ 0.004	551 $\pm$ 65	0.7828 $\pm$ 0.004	904.11 $\pm$ 84	0.6125 $\pm$ 0.002	3569 $\pm$ 62
Accurate 2L	0.9419 $\pm$ 0.003	8023 $\pm$ 55	0.9796 $\pm$ 0.005	570 $\pm$ 59	0.8189 $\pm$ 0.003	1085.37 $\pm$ 78	0.7312 $\pm$ 0.002	3622 $\pm$ 63
Efficient 2L	0.8321 $\pm$ 0.002	2584 $\pm$ 41	0.9020 $\pm$ 0.002	326 $\pm$ 55	0.7569 $\pm$ 0.007	485.23 $\pm$ 73	0.5965 $\pm$ 0.004	1810 $\pm$ 45
SaaN 2L w/o SaaN in Local GRL	0.9405 $\pm$ 0.002	3124 $\pm$ 89	0.969 $\pm$ 0.006	342 $\pm$ 71	0.8088 $\pm$ 0.002	666.41 $\pm$ 67	0.7025 $\pm$ 0.002	2034 $\pm$ 64
SaaN 2L w/o SaaN in Global GRL	0.9489 $\pm$ 0.002	6255 $\pm$ 65	0.9788 $\pm$ 0.004	540 $\pm$ 76	0.8141 $\pm$ 0.004	988.5 $\pm$ 45	0.7146 $\pm$ 0.001	3823 $\pm$ 79
SaaN 2L (proposed)	0.9535 $\pm$ 0.004	3254 $\pm$ 83	0.9783 $\pm$ 0.003	345 $\pm$ 80	0.8194 $\pm$ 0.001	672.13 $\pm$ 58	0.7201 $\pm$ 0.001	2042 $\pm$ 44

2L-GRL, which shows even lower accuracy than the base models, but it significantly reduces the learning time of Accurate 2L-GRL. As expected, our final model significantly reduces the learning times of the same model without SaaN in global GRL, which has to run GRL on the original input graph. Despite the slightly increased learning time compared to the same model without SaaN in local GRL, which is incurred by utilizing more nodes (i.e., SaaN super-nodes) in addition to the original nodes in each subgraph for local GRL, our model shows meaningful and consistent accuracy improvement.

E. Scalability

To evaluate the scalability of our model, we measure its performance compared to the two base models, GraphSAGE and GCN, by varying the number of CPU cores and dataset

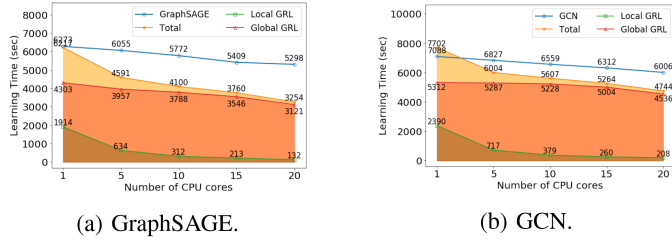


Fig. 9. Learning time changes as the number of CPU cores increase.

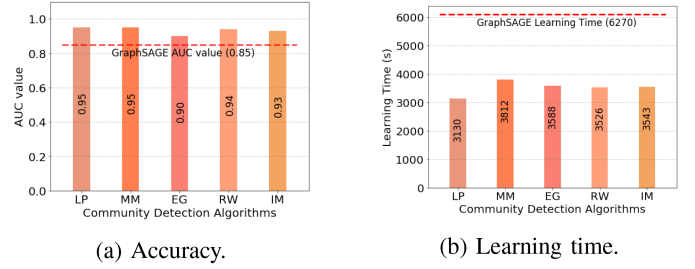


Fig. 11. Performance on diverse non-deterministic algorithms.

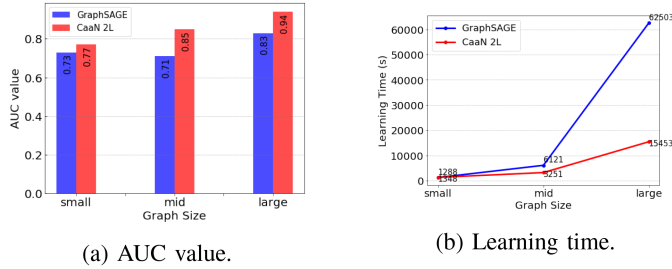


Fig. 10. Performance comparison as the scale of the dataset increases.

sizes. Fig. 9 shows the changes in learning time for link prediction on the Foursquare dataset as the number of CPU cores increases from 1 to 20. In addition to the total learning time, we also measure the local and global GRL times separately to provide further insights. We observe that, as the number of CPU cores increases, the total learning time consistently decreases. Specifically, with one CPU core, the total learning time of our model is comparable to that of the base models. The performance gains are negligible because our proposed model cannot utilize parallel processing for local GRL. However, with 20 cores, our model shows significantly larger gains (i.e., 21.01% ~ 38.58%). It is also worth noting that the rate of total time reduction is not proportional to the number of cores. This mainly stems from the fact that the effects of parallel processing in global GRL are limited as in the baseline models while local GRL decreases nearly proportionally to the number of CPU cores through parallel processing.

Fig. 10 shows the performance changes of our model with GraphSAGE as the base model on link prediction when we increase the dataset sizes using the YouTube datasets. Our model improves both the accuracy and learning efficiency for all dataset sizes, as shown in Fig. 10(a) and in Fig. 10(b), respectively. It is noteworthy that, as the graph size increases, so does the improvement of our model in learning accuracy compared to the base model. Specifically, our model improves the accuracy of small and large data by 5.47% and 13.25% compared to the base model, respectively. This indicates that our model takes advantage of dealing with large-scale graphs by effectively abstracting the entire graph with the SaaN graph. In terms of efficiency, our model shows a slight reduction of about 4.65% on small data, but a significant improvement of as much as 75.27% on large data. This is due to the high time complexity of the original GRL depending on the number of edges while our SaaN

TABLE IX
PERFORMANCE COMPARISON BASED ON DIVERSE PARTITIONING METHODS ON PUBMED DATASET

Partitioning Method (CD)		EG	MM	RW	LP	IM
# Subgraphs		20	42	852	977	1160
Partitioning Time (s)	CD	11.26	0.14	8.98	0.35	22.69
	Metis	24.58	35.93	749.37	825.26	OOM
	Libra	0.46	0.24	1.26	1.53	1.45
GRL Time (s)	Libra	21.71	30.74	1084.97	1483.99	1547.85
	CD	0.8291	0.8304	0.8421	0.8442	0.7925
	Metis	0.8073	0.7823	0.7682	0.7596	N/A
Accuracy (Baseline: 0.7828)	Libra with Rand	0.8204	0.8145	0.8130	0.7962	0.7960
	Libra with HighDeg	0.8218	0.8248	0.8136	0.7828	0.7938
	Libra with Avg	0.8103	0.8002	0.7976	0.7709	0.7750

graph mitigates this issue by effectively limiting the increase in the number of edges.

F. Performance on Diverse Partitioning Methods

We evaluate the effects of diverse non-deterministic partitioning algorithms in our model. Fig. 11 shows the performance of our model for link prediction on the Foursquare dataset using different community detection algorithms, compared to that of the base model, GraphSAGE (shown using red dashed lines). The results consistently demonstrate that our model improves accuracy with significantly reduced learning time regardless of the community detection algorithms. Specifically, our model shows higher AUC values with an improvement of 5.88% ~ 11.76% and reduces the learning time by 39.20% ~ 50.07% compared to that of the base model.

Finally, we extend our model to deterministic partitioning methods, i.e., the conventional edge-cut and vertex-cut algorithms. We utilize Metis [25] and Libra [26] as the representative edge-cut and vertex-cut partitioning methods, respectively, and implement them according to Algorithm 1 in Section V-D. We perform two kinds of experiments on two different datasets while maintaining the base model as GAT. First, to compare with the results of using non-deterministic partitioning algorithms, we set the number of partitions for deterministic partitioning equal to that obtained by community detection algorithms. Second, to directly compare the performance between Metis and Libra, we measure their performance as the number of partitions increases.

Table IX shows the performance of deterministic partitioning compared to non-deterministic partitioning in node classification on the Pubmed dataset. For each community detection algorithm, the number of subgraphs varies significantly: the Infomap algorithm (IM) produces 1160 subgraphs, while the

TABLE X
PERFORMANCE COMPARISON OF DETERMINISTIC PARTITIONING METHODS ON CITESEER DATASET

# Subgraphs	Metis			Libra		
	Accuracy	Partitioning Time (s)	GRL Time (s)	Accuracy	Partitioning Time (s)	GRL Time (s)
10	0.8825	0.01	35.26	0.7821	0.02	39.81
50	0.8997	0.08	150.09	0.8167	0.11	159.42
100	0.9033	0.11	308.67	0.8049	0.13	326.54
150	0.8918	0.12	479.16	0.7917	0.19	505.45
200	0.8746	0.10	658.05	0.8069	0.25	686.02
250	0.8788	0.08	926.24	0.8038	0.26	922.34
300	0.8601	0.10	1161.04	0.8221	0.28	1160.59

(Baseline accuracy: 0.62)

eigenvector-based (EG) method yields only 20 subgraphs. We also measure the performance of three variations of aggregation methods for a vertex-cut algorithm, Libra. Because they consistently show performance superiority compared to the baseline regardless of aggregation methods, we use Libra-Avg for the following experiment. Table X compares the performance of deterministic partitioning methods, i.e., edge-cut and vertex-cut when the number of partitions increases linearly. On a different dataset, CiteSeer, we compare two deterministic partitioning algorithms, Libra and Metis on node classification.

From both experiments, we find the following two observations. 1) Regarding accuracy, our SaaN 2L-GRL regardless of partitioning algorithms generally demonstrates higher accuracy compared to the baseline. In the case of Metis, a performance degradation compared to the baseline is observed when the number of subgraphs is large. However, because each deterministic algorithm can determine the optimal number of partitions, Metis also clearly outperforms the baseline when the number of subgraphs is determined as 20. 2) Regarding efficiency, for a smaller dataset, CiteSeer, all the partitioning methods show negligible partitioning overhead compared to GRL time. For a larger dataset, PubMed, Libra shows the fastest partitioning time, and community detection algorithms are generally efficient. However, it is worth noting that the partitioning time of Metis significantly increases for a large number of subgraphs. Here again, we can choose the optimal number of subgraphs for Metis in advance, e.g., 20 subgraphs significantly improve both learning efficiency and accuracy than 977 subgraphs in Table IX. We also note that the additional GRL time of Libra compared to Metis is not significant even though Libra includes processing for duplicated nodes in local GRL.

G. Effects of Condensing Node Attributes

In Definition V.1, we introduced condensing rules according to the type of node attributes when defining a super-node from the corresponding subgraph. To investigate the impact of node attributes of super-nodes on learning, we apply *AVG()* and *SUM()*, *MAX()*, *MIN()*, and *ZERO()* (i.e., initializing all attribute values to 0) to all the node attributes in each dataset and compare their performance with our method, *CONDENSE()*. Table XI presents the experimental results for link prediction tasks using GraphSAGE as the base model. *CONDENSE()* consistently outperforms the other possible condensing operators across all datasets. For the CiteSeer, Flickr, and PubMed datasets, where all attributes are numerical attributes, *AVG()* shows the best

TABLE XI
PERFORMANCE COMPARISON BY SUPER-NODE ATTRIBUTE CONDENSING METHODS

	Foursquare	CiteSeer	Flickr	PubMed
All <i>AVG()</i>	0.9535 ±0.004	0.8386 ±0.002	0.7457 0.001	0.8741 ±0.002
All <i>MAX()</i>	0.9523 ±0.002	0.8197 ±0.001	0.6728 ±0.001	0.8680 ±0.001
All <i>MIN()</i>	0.9492 ±0.004	0.7696 ±0.001	0.7399 ±0.001	0.8399 ±0.001
All <i>SUM()</i>	0.9497 ±0.001	0.8188 ±0.001	0.7312 ±0.001	0.8708 ±0.001
All <i>ZERO()</i>	0.9423 ±0.060	0.7557 ±0.001	0.6730 ±0.002	0.8367 ±0.001
<i>CONDENSE()</i>	0.9569 ±0.002	0.8386 ±0.002	0.7457 0.001	0.8741 ±0.002

performance according to our rule. *CONDENSE()* shows the best performance for the Foursquare dataset, where network and categorical attributes are mixed.

VII. CONCLUSIONS AND FUTURE WORK

In this study, we proposed a new GRL model, SaaN 2L-GRL, which improves both representation accuracy and learning efficiency of existing GRL models. Supporting with arbitrary graph partitioning, we perform GRL at two distinct levels: 1) GRL on nodes in each subgraph and 2) GRL on subgraphs. To effectively deal with the global patterns of the graph with minimal overhead, we introduced the Subgraph-as-a-Node Graph, which abstracts each subgraph as the representative node. Consequently, our proposed model significantly enhanced both the representation accuracy and learning efficiency of the existing GRL models. We demonstrated its effectiveness by applying it to two representative graph analysis tasks, i.e., link prediction and node classification, and evaluating its performance on five real-world graph datasets. We also verified that our model works any graph partitioning method, including community detection and conventional edge-cut and vertex-cut algorithms.

SaaN 2L-GRL was designed to leverage parallel processing, and its effectiveness was demonstrated through multi-processing on a single node. However, as the scale of graphs continues to increase, we must consider scalable methods for GRL in a distributed environment. In this regard, SaaN 2L-GRL has significant potential for processing graph data at a massive scale. As a future research direction, we plan to extend our SaaN 2L-GRL to a distributed environment. To achieve this, we need to investigate more sophisticated partitioning schemes that balance between nodes and enable effective replication across nodes, while also considering network costs between the nodes. In addition, to extend the applicability of our method to heterogeneous graphs, we plan to design a novel concept of SaaN graphs so as to represent various types of nodes and edges of the original graphs as super-nodes while effectively preserving information.

REFERENCES

- [1] L. Wu, P. Cui, J. Pei, L. Zhao, and L. Song, *Graph Neural Networks*. Berlin, Germany: Springer, 2022.

- [2] L. Wu et al., "Graph neural networks for natural language processing: A survey," *Found. Trends Mach. Learn.*, vol. 16, no. 2, pp. 119–328 2023.
- [3] Neo4j, "Graph database case studies." Accessed: Jul. 5, 2024. [Online]. Available: <https://neo4j.com/case-studies/>
- [4] S. Bonner et al., "A review of biomedical datasets relating to drug discovery: A knowledge graph perspective," *Brief. Bioinf.*, vol. 23, no. 6, 2022, Art. no. bbac404.
- [5] J. Zagalsky, "Graphs in government: The U.S. army," Neo4j. Accessed: Jul. 5, 2024. [Online]. Available: <https://neo4j.com/blog/graphs-in-government-the-u-s-army/>
- [6] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [7] P. Goyal and E. Ferrara, "Graph embedding techniques, applications, and performance: A survey," *Knowl.-Based Syst.*, vol. 151, pp. 78–94, 2018.
- [8] B. Li and D. Pi, "Network representation learning: A systematic literature review," *Neural Comput. Appl.*, vol. 32, no. 21, pp. 16647–16679, 2020.
- [9] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.
- [10] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.
- [11] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
- [12] C. Tu et al., "A unified framework for community detection and network representation learning," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 6, pp. 1051–1065, Jun. 2019.
- [13] N. T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [14] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [15] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv: 1710.10903*.
- [16] S. Cavallari, V. W. Zheng, H. Cai, K. C.-C. Chang, and E. Cambria, "Learning community embedding with community detection and node embedding on graphs," in *Proc. ACM Conf. Inf. Knowl. Manage.*, 2017, pp. 377–386.
- [17] X. Wang, P. Cui, J. Wang, J. Pei, W. Zhu, and S. Yang, "Community preserving network embedding," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 203–209.
- [18] J. Wang, J. Cao, W. Li, and S. Wang, "Cane: Community-aware network embedding via adversarial training," *Knowl. Inf. Syst.*, vol. 63, no. 2, pp. 411–438 2021.
- [19] Y. Liu, Z. Liu, X. Feng, and Z. Li, "Robust attributed network embedding preserving community information," in *Proc. IEEE 38th Int. Conf. Data Eng.*, 2022, pp. 1874–1886.
- [20] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Phys. Rev. E*, vol. 76, no. 3, 2007, Art. no. 036106.
- [21] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of community hierarchies in large networks," 2008, *arXiv:0803.0476*.
- [22] M. E. J. Newman, "Finding community structure in networks using the eigenvectors of matrices," *Phys. Rev. E*, vol. 74, no. 3, 2006, Art. no. 036104.
- [23] P. Pons and M. Latapy, "Computing communities in large networks using random walks," in *Proc. Comput. Inf. Sci.: 20th Int. Symp.*, 2005, pp. 284–293.
- [24] M. Rosvall and C. T. Bergstrom, "Maps of information flow reveal community structure in complex networks," 2007, *arXiv:0707.0609*.
- [25] G. Karypis and V. Kumar, "METIS: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices," Univ. Digit. Conservancy, 1997. [Online]. Available: <https://hdl.handle.net/11299/215346>
- [26] C. Xie, L. Yan, W.-J. Li, and Z. Zhang, "Distributed power-law graph computing: Theoretical and empirical analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 1–9.
- [27] H. Cai, V. W. Zheng, and K. C.-C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018.
- [28] Y. Lin, M. Yang, J. Yu, P. Hu, C. Zhang, and X. Peng, "Graph matching with bi-level noisy correspondence," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 23362–23371.
- [29] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph matching networks for learning the similarity of graph structured objects," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3835–3845.
- [30] F. Xia, J. Liu, H. Nie, Y. Fu, L. Wan, and X. Kong, "Random walks: A review of algorithms and applications," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 4, no. 2, pp. 95–107, Apr. 2020.
- [31] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [32] L. Ruiz, F. Gama, and A. Ribeiro, "Gated graph recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 68, pp. 6303–6318, 2020.
- [33] H. Dai, Z. Kozareva, B. Dai, A. Smola, and L. Song, "Learning steady-states of iterative algorithms over graphs," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1106–1114.
- [34] N. T. Kipf and M. Welling, "Variational graph auto-encoders," 2016, *arXiv:1611.07308*.
- [35] S. Pan, R. Hu, G. Long, J. Jiang, L. Yao, and C. Zhang, "Adversarially regularized graph autoencoder for graph embedding," 2018, *arXiv:1802.04407*.
- [36] J. You, R. Ying, X. Ren, W. Hamilton, and J. Leskovec, "GraphRNN: Generating realistic graphs with deep auto-regressive models," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5708–5717.
- [37] J. Liu, A. Kumar, J. Ba, J. Kiro, and K. Swersky, "Graph normalizing flows," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019.
- [38] Z. Peng, Y. Dong, M. Luo, X.-M. Wu, and Q. Zheng, "Self-supervised graph representation learning via global context prediction," 2020, *arXiv:2003.01604*.
- [39] F. Chen, Y.-C. Wang, B. Wang, and C.-C. J. Kuo, "Graph representation learning: A survey," *APSIPA Trans. Signal Inf. Process.*, vol. 9, 2020, Art. no. e15.
- [40] W. Hu et al., "Strategies for pre-training graph neural networks," 2019, *arXiv:1905.12265*.
- [41] Y. Rong et al., "Self-supervised graph transformer on large-scale molecular data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 12559–12571.
- [42] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," 2018, *arXiv:1809.10341*.
- [43] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph contrastive learning with adaptive augmentation," in *Proc. Web Conf.*, 2021, pp. 2069–2080.
- [44] Y. Liu, Y. Zheng, D. Zhang, H. Chen, H. Peng, and S. Pan, "Towards unsupervised deep graph structure learning," in *Proc. ACM Web Conf.*, 2022, pp. 1392–1403.
- [45] Y. Wang et al., "Clusterscl: Cluster-aware supervised contrastive learning on graphs," in *Proc. ACM Web Conf.*, 2022, pp. 1611–1621.
- [46] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 1150–1160.
- [47] Q. Sun et al., "Sugar: Subgraph neural network with reinforcement pooling and self-supervised mutual information mechanism," in *Proc. Web Conf.*, 2021, pp. 2081–2091.
- [48] D. Kim and A. Oh, "Efficient representation learning of subgraphs by subgraph-to-node translation," 2022, *arXiv:2204.04510*.
- [49] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," *Adv. Neural Inf. Process. Syst.*, 2018, pp. 4805–4815.
- [50] Y. He et al., "Distmle: A distributed multi-level framework for scalable graph embedding," in *Proc. IEEE 28th Int. Conf. High Perform. Comput., Data, Analytics*, 2021, pp. 282–291.
- [51] J. Liang, S. Gurukar, and S. Parthasarathy, "Mile: A multi-level framework for scalable graph embedding," in *Proc. Int. AAAI Conf. Web Social Media*, 2021, pp. 361–372.
- [52] D. Yang, J. Liu, and J. Lai, "Edges: An efficient distributed graph embedding system on GPU clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1892–1902, Jul. 2021.
- [53] C. Böhm and C. Plant, "Massively parallel graph drawing and representation learning," in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 609–616.
- [54] J. Qiu et al., "NetSMF: Large-scale network embedding as sparse matrix factorization," in *Proc. World Wide Web Conf.*, 2019, pp. 1509–1520.
- [55] J. Zhang, Y. Dong, Y. Wang, J. Tang, and M. Ding, "ProNE: Fast and scalable network representation learning," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4278–4284.
- [56] K. Andreev and H. Räcke, "Balanced graph partitioning," in *Proc. 16th Annu. ACM Symp. Parallelism Algorithms Architectures*, 2004, pp. 120–124.

- [57] F. Bourse, M. Lelarge, and M. Vojnovic, "Balanced graph edge partition," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 1456–1465.
- [58] S. Ji, C. Bu, L. Li, and X. Wu, "Local graph edge partitioning with a two-stage heuristic method," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 228–237.
- [59] V. Md et al., "DistGNN: Scalable distributed training for large-scale graph neural networks," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2021, pp. 1–14.
- [60] D. Kong, X. Xie, and Z. Zhang, "Clustering-based partitioning for large web graphs," in *Proc. IEEE 38th Int. Conf. Data Eng.*, 2022, pp. 593–606.
- [61] A. C. Zhou, J. Luo, R. Qiu, H. Tan, B. He, and R. Mao, "Adaptive partitioning for large-scale graph analytics in GEO-distributed data centers," in *Proc. IEEE 38th Int. Conf. Data Eng.*, 2022, pp. 2818–2830.
- [62] H. Yin, M. Zhang, Y. Wang, J. Wang, and P. Li, "Algorithm and system co-design for efficient subgraph-based graph representation learning," 2022, *arXiv:2202.13538*.
- [63] L. Wu, Q. Zhang, C.-H. Chen, K. Guo, and D. Wang, "Deep learning techniques for community detection in social networks," *IEEE Access*, vol. 8, pp. 96016–96026, 2020.
- [64] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," in *Proc. Nat. Acad. Sci.*, vol. 99, no. 12, pp. 7821–7826, 2002.
- [65] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters," *Internet Math.*, vol. 6, no. 1, pp. 29–123 2009.
- [66] W. Li, J. Xie, M. Xin, and J. Mo, "An overlapping network community partition algorithm based on semi-supervised matrix factorization and random walk," *Expert Syst. Appl.*, vol. 91, pp. 277–285, 2018.
- [67] J.-H. Park and H.-Y. Kwon, *Cyberattack Detection Model Using Community Detection and Text Analysis on Social Media*. West Midlands, U.K.: ICT Express, 2021.
- [68] L. Jiang, L. Shi, L. Liu, J. Yao, and M. E. Ali, "User interest community detection on social media using collaborative filtering," *Wireless Netw.*, vol. 28, pp. 1169–1175, 2022.
- [69] Y. Wu, Y. Fu, J. Xu, H. Yin, Q. Zhou, and D. Liu, "Heterogeneous question answering community detection based on graph neural network," *Inf. Sci.*, vol. 621, pp. 652–671, 2023.
- [70] J.-H. Park, K. Lee, and H.-Y. Kwon, "Two-level graph representation learning with community-as-a-node graphs," in *Proc. IEEE Int. Conf. Data Mining*, 2023, pp. 1259–1264.
- [71] X. Huang, J. Li, and X. Hu, "Label informed attributed network embedding," in *Proc. 10th ACM Int. Conf. Web Search Data Mining*, 2017, pp. 731–739.
- [72] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Mag.*, vol. 29, no. 3, pp. 93–93, 2008.
- [73] C. L. Giles, K. D. Bollacker, and S. Lawrence, "CiteSeer: An automatic citation indexing system," in *Proc. 3rd ACM Conf. Digit. Libraries*, 1998, pp. 89–98.
- [74] P. Mernyei and C. Cangea, "Wiki-CS: A wikipedia-based benchmark for graph neural networks," 2020, *arXiv: 2007.02901*.
- [75] O. Shchur, M. Mummé, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," 2018, *arXiv: 1811.05868*.
- [76] A. Mislove, M. Marcon, K. P. Gummadi, P. Druschel, and B. Bhattacharjee, "Measurement and analysis of online social networks," in *Proc. 5th ACM/Usenix Internet Meas. Conf.*, 2007, pp. 29–42.
- [77] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36 1982.



Jeong-Ha Park received the MS degree in data science from the Seoul National University of Science and Technology (SeoulTech), in 2023. Her research interests include Applied Machine Learning, specifically graph representation learning and graph neural networks.



Bo-Young Lim is currently working toward the MS degree in data science from the Seoul National University of Science and Technology (SeoulTech). Her research interests include graph representation learning, distributed graph processing, dynamic graph learning, and continual learning.



Kisung Lee received the bachelor's and master's degrees in computer science from KAIST, and the PhD degree in computer science from the Georgia Institute of Technology (Georgia Tech). He is currently an associate professor with the Division of Computer Science and Engineering, Louisiana State University (LSU). His research interests include the intersection of Big Data and distributed computing systems. He is also working on research problems in spatial data management and social media analytics.



Hyuk-Yoon Kwon (Member, IEEE) received the PhD degree in computer science from KAIST, in 2013. He is an associate professor with the ITM Division, Department of Industrial Engineering / Graduate School of Data Science, Seoul National University of Science and Technology (SeoulTech). His research interests include data-driven AI/ML, Big Data management, distributed & cloud computing, federated & distributed learning, databases, data-centric cybersecurity, and fair data scraping and analysis.